

Zakura Broadcast Gateway Plaintext Backend Relay Hijacking Vulnerability Report

Summary

The broadcast gateway can forward transactions to remote backends over plaintext HTTP without backend authentication. Because a 200 response is not validated as a genuine JSON-RPC success, an on-path attacker can read or suppress transactions and forge success so the client believes a broadcast occurred.

Mitigating Factors:

- The attacker must control or observe the network path between the gateway and a remote backend.
- Loopback backends, a trusted private tunnel, or mutually authenticated TLS removes the exposed plaintext hop.
- Frontend HTTPS still protects the client-to-gateway leg; the weakness is the separate gateway-to-backend leg.
- Strict egress routing and backend firewall rules reduce the chance of path manipulation.

Aggravating Factors:

- Repository configurations include cross-host plaintext http:// backend URLs.
- The loader checks the URL scheme but does not require encryption or a loopback destination.
- Backend requests carry no Authorization header and use no mutual TLS identity.
- BackendPool.forward treats HTTP 200 as usable without validating JSON-RPC success, response id, or a 64-character hexadecimal transaction id.
- A forged 200 response prevents failover to a healthy backend.

Vulnerability Details:

Status, Product, and Version

- Record status: PUBLISHED
- Product and version: Zakura / 1.2.0
- Weakness classification: CWE-319: Cleartext Transmission of Sensitive Information; CWE-345: Insufficient Verification of Data Authenticity

Cross-Validation Conclusion

The risk is valid for public or otherwise attacker-observable plaintext backend routes. Client-side HTTPS does not authenticate the next hop, and response-shape validation is insufficient to distinguish a real node response from an injected success.

Trigger Conditions

- At least one gateway backend uses plaintext HTTP across a non-loopback network path.
- The backend does not require a shared credential or mutual TLS.
- The attacker can intercept, redirect, or inject traffic on that path.
- The gateway accepts the forged 200 response without semantic validation or failover.

Normal and Abnormal Behavior

Normal behavior

The gateway authenticates the backend, verifies transport integrity, validates the JSON-RPC envelope and txid, and fails over when any check fails.

Abnormal behavior

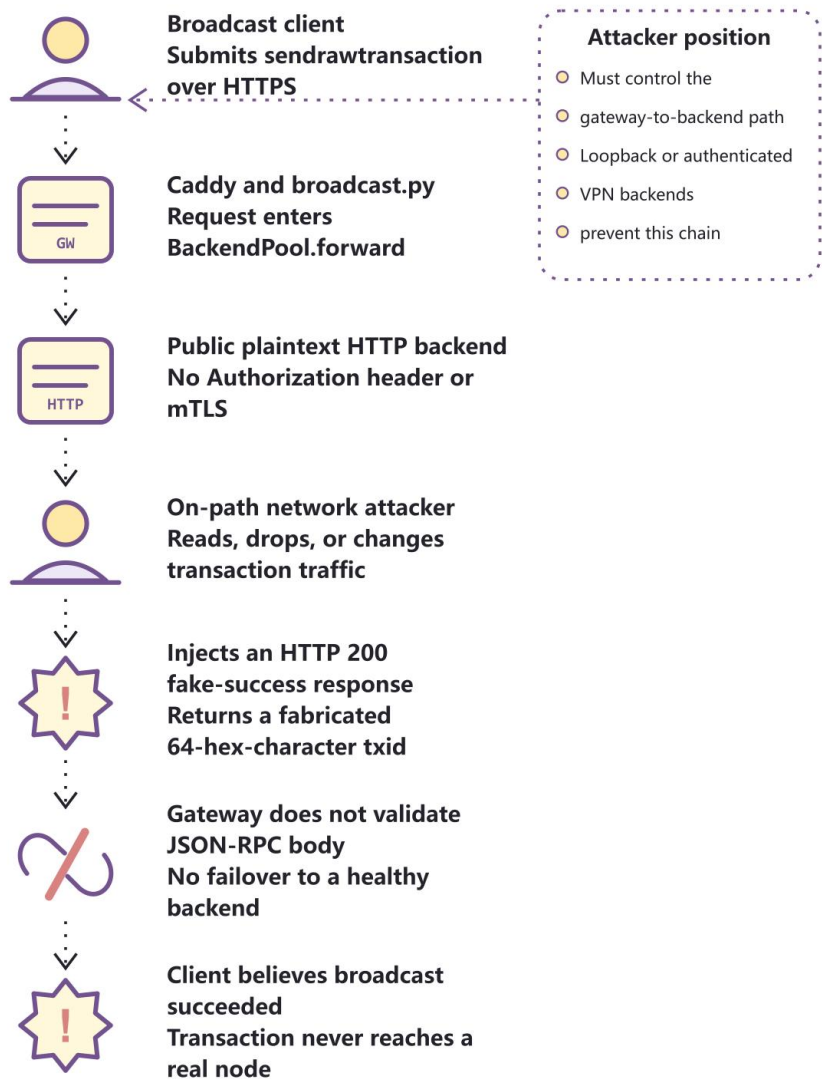
A network attacker returns an arbitrary HTTP 200 body that the gateway relays as success even though no real node accepted the transaction.

Full Call Chain Diagram

The complete call chain is shown below. The dashed box states the deployment condition, version boundary, or evidence boundary that controls the conclusion.

Broadcast Gateway Relay-Hijacking Call Chain

Frontend HTTPS cannot protect a plaintext public backend hop



Technical Details:

Root cause 1: remote backends may use plaintext HTTP

```
[[backends]]
url = "http://remote-backend.example:8232/"
# No authenticated transport is required.
```

The scheme and destination allow transaction contents and responses to cross an unauthenticated network path.

Root cause 2: no backend identity is presented or verified

Requests do not carry an RPC cookie or gateway-specific token, and the connection does not use mutual TLS.

Root cause 3: HTTP status substitutes for response authenticity

```
response = session.post(backend.url, data=body)
if response.status_code < 500:
    return response.content
# JSON-RPC error, id, and txid shape are not validated here.
```

Minimal Reproduction Logic (Isolated Test Environment Only)

In an isolated gateway test, point the first backend at a controlled server that returns HTTP 200 with a fake txid or JSON-RPC error. Confirm whether the gateway relays it and whether a known healthy second backend is called.

[illegible]

Security Impact

- An on-path attacker can observe transaction contents sent to a remote backend.
- The attacker can suppress or replace relay traffic and selectively prevent broadcast.
- A forged success can cause clients and automation to stop retrying even though the transaction never reached a node.

Long-Term Remediation

- Require mutually authenticated TLS, a trusted VPN, or an authenticated tunnel for every cross-host backend.
- Reject non-loopback plaintext HTTP backends during configuration loading.
- Validate the full JSON-RPC envelope, matching id, absence of error, and a 64-character hexadecimal txid before accepting success.
- Fail over on semantic validation failure, not only transport exceptions or 5xx status codes.

Recommended Regression Tests:

- Reject a non-loopback http:// backend unless an explicit secure tunnel profile is active.
- Return 200 with a JSON-RPC error and verify failover.
- Return 200 with malformed JSON, the wrong id, or an invalid txid and verify failover.
- Verify backend certificate and client-identity failures are reported and never treated as broadcast success.